

Chapter 13. CI/CD로 품질을 지속시킨다

이 챕터에서 배우는 것

- 사람이 수동으로 `book-forge gate` 를 실행하는 방식이 왜 결국 빠지는 순간을 만드는지
- `gate_cmd.py` 에 이미 있는 CI 연동 플래그(`--save-baseline` · `--fail-on-regression` · `--golden-set` · `--junit-xml`) 전체
- WARN/FAIL이 나왔을 때 무엇부터 고칠지 정하는 우선순위
- 처음부터 강하게 막지 않는 점진적 하드페일 롤아웃
- Book-forge가 아직 하지 않는 것 — 추세 회귀 감지와 골든셋 자동 추출

이런 분이 먼저 읽으면 좋습니다: 12장에서 `book-forge gate` 로 책 전체를 판정하는 법을 봤는데, 이걸 매번 손으로 실행하는 대신 자동화하고 싶은 분.

13.1 문제 — 사람이 안 돌리면 아무도 모른다

12장까지 다룬 `book-forge gate` 는 저자가 터미널에서 직접 실행해야 하는 명령이다. 이 방식의 문제는 명확하다. **사람이 실행을 깜빡하는 순간, 그 순간의 품질 하락은 아무에게도 보고되지 않는다.** 챕터를 급하게 고치고 커밋만 하고 게이팅을 건너뛰거나, 팀원이 새로 합류해 이 관례를 모르거나, 의존성이 조용히 업그레이드돼 프롬프트 출력 형식이 바뀌는 경우가 전부 이 경로로 새어나간다. Agent-Evaluator 실전 이식 가이드가 이 문제를 한 문장으로 요약한다. **"측정이 사람의 행동과 독립적으로 실행되어야 한다."** 이 챕터는 12장까지 다룬 `book-forge gate` 를 사람 손을 거치지 않고 자동으로 돌리는 방법을 다룬다.

13.2 이미 있는 코드 — `gate_cmd.py` 의 CI 연동 플

래그

새로 만들 것은 거의 없다. `gate_cmd.py` 는 `agent-eval gate --help` 전체를 그대로 노출한다는 원칙(모듈 docstring)에 따라, CI 연동에 필요한 플래그가 이미 다 배선돼 있다.

 파일: `src/book_forge/cli/commands/gate_cmd.py`

```
@click.option("--baseline", type=click.Path(), default=None, help="baseline 파일  
경로 (기본: <결과폴더>/baseline.json)")  
@click.option("--baseline-version", default=None, help="버전별 독립 baseline 태그")  
@click.option("--save-baseline", is_flag=True, help="현재 결과를 baseline으로 저장")  
@click.option("--fail-on-regression", type=float, default=None, help="baseline  
대비 허용 회귀(%) 초과 시 exit 2")  
@click.option("--golden-set", type=click.Path(), default=None, help="골든셋 JSON  
경로")  
@click.option("--fail-on-golden-regression", is_flag=True, help="골든셋 케이스 누락/  
실패 시 exit 3")  
@click.option("--junit-xml", default=None, help="JUnit XML 출력 경로 (CI 연동용)")
```

각 플래그가 CI에서 실제로 하는 역할은 다음과 같다.

플래그	역할	언제 쓰는가
<code>--save-baseline</code>	지금 이 실행 결과를 "기준선"으로 저장	품질이 만족스러운 시점(예: 릴리스 직전)에 1회 실행
<code>--fail-on-regression <퍼센트></code>	baseline 대비 이 비율 이상 나빠지면 exit 2	매 CI 실행마다 — 절대 점수가 아니라 "이전보다 나빠졌는가"를 본다
<code>--golden-set <경로> + --fail-on-golden-regression</code>	미리 정해진 대표 사례들이 여전히 통과하는지 확인, 실패 시 exit 3	회귀 테스트처럼 반복 실행
<code>--junit-xml <경로></code>	결과를 JUnit XML로 출력	GitHub Actions 등이 테스트 결과 탭에 그대로 표시하게 함

exit code는 `agent-eval` CLI가 정한 그대로 전달된다(`gate()`의 docstring). **0=통과, 1=미달, 2=baseline 대비 회귀, 3=골든셋 회귀**. CI 스텝은 이 네 가지를 구분해 각기 다른 알림을 보낼 수 있다.

13.3 GitHub Actions에 실제로 연결하기

 이 책이 작성한 예시 워크플로 (Book-forge/Agent-Evaluator 저장소에는 실재하지 않음 — CI 연동 방법을 보여주기 위해 이 책이 직접 작성했다)

```
name: book-forge-gate
on:
  pull_request:
    paths: ["Book/**", "src/book_forge/**"]

jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -e ".[dev]"
      - name: Gate A-G 판정 (책 전체 집계)
        run: |
          book-forge gate "내_프로젝트" \
            --min-gate-score 0.6 \
            --fail-on-regression 10 \
            --junit-xml gate-report.xml
      - uses: dorny/test-reporter@v1
        if: always()
        with:
          name: "Book-forge Gate 결과"
          path: gate-report.xml
          reporter: java-junit
```

이 워크플로는 새 아키텍처가 아니다. 12장에서 이미 확인한 `book-forge gate` 명령 하나를 그대로 CI 러너 위에서 실행할 뿐이다. **CI 통합의 핵심은 새 도구가 아니라, 사람이 하던 것을 사람 개입 없이 매번 실행하는 것뿐이다.**

13.4 WARN/FAIL이 나왔을 때 — 무엇부터 고칠 것인가

CI가 여러 Gate에서 동시에 경고를 내면, 전부를 한꺼번에 고치려다 아무것도 못 고치는 경우가 생긴다. Agent-Evaluator 실전 이식 가이드가 제시하는 우선순위를, Book-forge 같은 1인~소규모 프로젝트 규모에 맞게 정리하면 다음과 같다.

우선순위	상황	대응 시점
1(즉시)	Gate E(보안) FAIL	바로 — 프롬프트 인젝션·PII 유출은 되돌리기 가장 어렵다
2(근시일)	Gate A(목표 달성) FAIL	다음 작업 세션 내 — 이 도구의 핵심 가치와 직결된다
3(단기)	Gate B(반복)·Gate D(성능) WARN	다음 며칠 내 — 비용·시간 낭비로 이어질 수 있다
4(여유 있을 때)	나머지 WARN	다음에 그 챕터를 다시 만질 때 함께

모든 Gate를 한 번에 PASS로 만들려 하지 않는다. Agent-Evaluator 실전 이식 가이드의 표현을 빌리면, "Gate 리포트가 지속적으로 나오는 환경을 만드는 것이 먼저다." 리포트 자체가 없으면 우선순위를 정할 근거조차 없다.

13.5 점진적 하드페일 롤아웃 — 처음부터 세계 막지 않는다

CI에 게이팅을 처음 연결하는 순간부터 `--fail-on-regression` 을 하드 실패로 걸면, 아직 기준선이 안정되지 않은 초기에는 사소한 변동에도 PR이 막혀버릴 수 있다. 10장 (§ 10.4)이 가중치에 대해 말한 것과 같은 원칙이 CI 단계에도 그대로 적용된다. 처음에는 아래처럼 **경고만 하고 막지는 않는** 단계를 거친 뒤, 팀이 기준에 익숙해지면 강제로 전환한다.

 이 책이 작성한 예시 워크플로 (저장소에는 실재하지 않음)

```
- name: Gate A-G 판정 (아직 경고만, 병합은 막지 않음)
  run: book-forge gate "내_프로젝트" --min-gate-score 0.6
  continue-on-error: true # 익숙해지면 이 줄부터 지운다
```

`continue-on-error: true` 를 지우는 순간이 "이 프로젝트가 게이팅을 실제로 신뢰하기 시작한 시점"이라고 봐도 된다.

13.6 정직한 한계 — Book-forge가 아직 안 하는 것

Agent-Evaluator SDK는 `--fail-on-regression` (§ 13.2, 단일 실행 대비 회귀) 외에도 여러 실행에 걸친 추세를 보는 `agent-eval trend` (예: `--window 8` 로 최근 8회 실행의 기울기를 계산해, 매번은 기준을 통과해도 서서히 나빠지는 추세를 감지)와, 결과 파일에서 대표 사례를 자동으로 뽑아주는 `GoldenSetBuilder` 를 함께 제공한다. Book-forge 소스 전체를 검색해보면(`grep -rln "trend\\|GoldenSetBuilder" src/book_forge/`), 이 둘은 어디에도 배선돼 있지 않다. `--golden-set` (§ 13.2)은 CLI 플래그로만 노출돼 있을 뿐, 그 골든셋 JSON을 자동으로 만들어주는 경로가 Book-forge에는 없다 — 저자가 직접 대표 사례를 골라 파일로 만들어야 한다.

이 공백을 숨기지 않고 여기 남긴다(15장 § 15.5가 이미 지켜온 원칙과 같은 자리다). "매 실행은 통과하는데 몇 주에 걸쳐 서서히 나빠지는 품질"은 지금의 Book-forge로는 사람이 직접 알아채야 한다. 이 기능을 실제로 배선하려는 독자를 위한 출발점은 17장(한계와 열린 문제)에 이어서 정리해줬다.

직접 해보기

`book-forge gate <slug> --save-baseline` 을 한 번 실행해 기준선을 저장한 뒤, 챕터 하나를 일부러 짧게 잘라 품질을 낮춰보고 `book-forge gate <slug> --fail-on-regression 5` 를 다시 실행해보라. exit code가 2로 바뀌는 것(§ 13.2의 표)을 직접 확인할 수 있다. 여러분의 프로젝트에 CI가 있다면, § 13.3의 워크플로를 `continue-on-error: true` 를 켜 채로 먼저 붙여보고, 며칠 지켜본 뒤 그 줄을 지우는 § 13.5의 점진적 전환을 실제로 따라가보라.

이 챕터의 핵심

- **측정은 사람의 행동과 독립적으로 실행되어야 한다.** 수동 실행은 반드시 빠지는 순간을 만든다.
- **CI 연동에 필요한 플래그는 이미 다 있다.** `--save-baseline` / `--fail-on-regression` / `--golden-set` / `--junit-xml` 모두 `gate_cmd.py` 가 `agent-eval gate` 를 그대로 위임해 얻은 것이다.
- **WARN/FAIL이 여럿이면 우선순위부터 정한다.** 보안(Gate E)이 항상 먼저다.
- **하드페일은 점진적으로 도입한다.** `continue-on-error: true` 로 시작해 팀이 익숙해지면 지운다.
- **추세 회귀 감지와 골든셋 자동 추출은 Book-forge에 아직 없다.** 있는 척하지 않는다 — 17장이 이 목록을 이어받는다.

참고 자료

- `src/book_forge/cli/commands/gate_cmd.py` — 전체(모든 CI 플래그의 실제 배선)
- `Agent-Evaluator/Media/Book/Part_VI_실전이식가이드/Chapter_27_CICD_완성.md` — 이 챕터가 재구성한 원본 방법론(추세 분석·골든 데이터셋 자동화 포함)
- 17장(한계와 열린 문제) — `agent-eval trend` / `GoldenSetBuilder` 미배선 항목

Part IV는 배치 평가(Gate A-G)와 완전히 다른 축 — **부작용이 있는 동작을 실행 전에 막는** 실시간 가드레일로 넘어간다.